

Дз 1. Лямбда-исчисление

Глобальный минимум: 86.

Блок 1. Синтаксис и семантика лямбда-исчисления

Локальный минимум: 46.

Задание 1.1 (16)

Уберите все лишние скобки из выражений.

Подсказка: можно воспользоваться представлением терма как дерева.

a) $(\lambda x. (x (x)))$

Решение:

$\lambda x. x x$

b) $(\lambda x y. f ((g x) y))$

Решение:

$\lambda x y f. f (g x y)$

c) $x (\lambda y. (\lambda z. x y z))$

Решение:

$x \lambda y z. x y z$

Задание 1.2 (16)

Найдите все свободные переменные, а также обозначьте, к какому связывателю относится каждая связанная (это удобнее всего сделать с помощью индексов).

a) $\lambda x. \lambda x. x y$

Решение:

Свободные: y
 $\lambda x^0. \lambda x^1. x^1 y$

b) $\lambda z. (\lambda x. x) (x z)$

Решение:

Свободные: x
 $\lambda z^0. (\lambda x^1. x^1) (x z^0)$

с) $(\lambda x. y \ y \ (\lambda y. x \ ((\lambda z. z \ y \ x) \ (\lambda x. z \ x))))$

Решение:

Свободные: y, z

$(\lambda x^0. y \ y \ (\lambda y^1. x^0 \ ((\lambda z^2. z^2 \ y^1 \ x^0) \ (\lambda x^3. z \ x^3))))$

Задание 1.3

а) (16) Покажите, что $\mathbf{K}^* =_{\beta} \mathbf{K} \mathbf{I}$.

Решение:

$$\mathbf{K} \mathbf{I} \equiv (\lambda x \ y. x) \lambda x. x \rightarrow_{\beta} \quad (1)$$

$$\rightarrow_{\beta} \lambda y. \lambda x. x =_{\alpha} \quad (2)$$

$$=_{\alpha} \lambda x \ y. y \equiv \mathbf{K}^* \quad (3)$$

б) (16) Покажите, что $\mathbf{B} =_{\beta} \mathbf{S} (\mathbf{K} \mathbf{S}) \mathbf{K}$, расписывая и нумеруя промежуточные шаги.

Подсказка: не спешите сразу раскрывать все определения.

Решение:

$$\mathbf{S} (\mathbf{K} \mathbf{S}) \mathbf{K} \equiv (\lambda f \ g \ x. f \ x \ (g \ x)) (\mathbf{K} \mathbf{S}) \mathbf{K} \rightarrow_{\beta} \quad (4)$$

$$\rightarrow_{\beta} (\lambda g \ x. (\mathbf{K} \mathbf{S}) \ x \ (g \ x)) \mathbf{K} \rightarrow_{\beta} \quad (5)$$

$$\rightarrow_{\beta} \lambda x. (\mathbf{K} \mathbf{S}) \ x \ (\mathbf{K} \ x) \equiv \quad (6)$$

$$\equiv \lambda x. ((\lambda x \ y. x) \mathbf{S}) \ x \ ((\lambda x \ y. x) \ x) \rightarrow_{\beta} \quad (7)$$

$$\rightarrow_{\beta} \lambda x. (\lambda y. \mathbf{S}) \ x \ (\lambda y. x) \rightarrow_{\beta} \quad (8)$$

$$\rightarrow_{\beta} \lambda x. \mathbf{S} \ (\lambda y. x) \equiv \quad (9)$$

$$\equiv \lambda x^0. (\lambda f \ g \ x^1. f \ x^1 \ (g \ x^1)) (\lambda y. x^0) \rightarrow_{\beta} \quad (10)$$

$$\rightarrow_{\beta} \lambda x^0. \lambda g \ x^1. (\lambda y. x^0) \ x^1 \ (g \ x^1) \rightarrow_{\beta} \quad (11)$$

$$\rightarrow_{\beta} \lambda x^0. \lambda g \ x^1. x^0 \ (g \ x^1) =_{\alpha} \quad (12)$$

$$=_{\alpha} \lambda f \ g \ x. f \ (g \ x) \equiv \mathbf{B} \quad (13)$$

Задание 1.4

Осуществите подстановку и приведите к нормальной форме — форме, в которой не осталось редексов, указывая при этом основные шаги.

а) (16) $x := \mathbf{S} \ \mathbf{B} \ x \ (\lambda z \ x. z \ x) \ (\lambda z \ y \ x. x) \ x.$

Решение:

$$\mathbf{S} (\lambda z x. z x) (\lambda z y x. x) \mathbf{S} \equiv \quad (14)$$

$$\equiv (\lambda f g x. f x (g x)) (\lambda z x. z x) (\lambda z y x. x) \mathbf{S} \rightarrow_{\beta} \quad (15)$$

$$\rightarrow_{\beta} (\lambda z x. z x) \mathbf{S} ((\lambda z y x. x) \mathbf{S}) \rightarrow_{\beta} \quad (16)$$

$$\rightarrow_{\beta} \mathbf{S} ((\lambda z y x. x) \mathbf{S}) \equiv \quad (17)$$

$$\equiv (\lambda f g x. f x (g x)) ((\lambda z y x. x) \mathbf{S}) \rightarrow_{\beta} \quad (18)$$

$$\rightarrow_{\beta} \lambda g x. ((\lambda z y x. x) \mathbf{S}) x (g x) \rightarrow_{\beta} \quad (19)$$

$$\rightarrow_{\beta} \lambda g x. g x \quad (20)$$

b) (16) $x := \mathbf{K}$ и $y := \mathbf{B}$ в $y (\mathbf{K} (\lambda z. x x (x z x)) y) (\lambda x. x x x) \mathbf{I}$.

Решение:

$$\mathbf{B} (\mathbf{K} (\lambda z. \mathbf{K} \mathbf{K} (\mathbf{K} z \mathbf{K})) \mathbf{B}) (\lambda x. x x x) \mathbf{I} \equiv \quad (21)$$

$$\equiv (\lambda f g x. f (g x)) (\mathbf{K} (\lambda z. \mathbf{K} \mathbf{K} (\mathbf{K} z \mathbf{K})) \mathbf{B}) (\lambda x. x x x) \mathbf{I} \rightarrow_{\beta} \quad (22)$$

$$\rightarrow_{\beta} (\mathbf{K} (\lambda z. \mathbf{K} \mathbf{K} (\mathbf{K} z \mathbf{K})) \mathbf{B}) ((\lambda x. x x x) \mathbf{I}) \equiv \quad (23)$$

$$\equiv ((\lambda x y. x) (\lambda z. \mathbf{K} \mathbf{K} (\mathbf{K} z \mathbf{K})) \mathbf{B}) ((\lambda x. x x x) \mathbf{I}) \rightarrow_{\beta} \quad (24)$$

$$\rightarrow_{\beta} (\lambda z. \mathbf{K} \mathbf{K} (\mathbf{K} z \mathbf{K})) ((\lambda x. x x x) \mathbf{I}) \rightarrow_{\beta} \quad (25)$$

$$\rightarrow_{\beta} \mathbf{K} \mathbf{K} (\mathbf{K} ((\lambda x. x x x) \mathbf{I}) \mathbf{K}) \rightarrow_{\beta} \quad (26)$$

$$\rightarrow_{\beta} \mathbf{K} \quad (27)$$

Блок 2. Программирование в лямбда-исчислении

Локальный минимум: 3б.

Задание 2.1 (16)

Задайте лямбда-терм **or** $b_1 b_2$, который вычисляет дизъюнкцию двух логических величин.

Решение:

or $:= \lambda l r. l \text{ true } r$

Примеры:

$$\text{or true false} \rightarrow_{\beta} \text{true true false} \rightarrow_{\beta} (\lambda x. \text{true}) \text{false} \rightarrow_{\beta} \text{true} \quad (28)$$

$$\text{or false true} \rightarrow_{\beta} \text{false true false} \rightarrow_{\beta} (\lambda x. x) \text{true} \rightarrow_{\beta} \text{true} \quad (29)$$

Задание 2.2 (16)

Задайте лямбда-терм **mult** $a b$, перемножающий два числа Чёрча, и лямбда-терм **pow** $a b$, который вычисляет a^b .

Решение:**mult** := $\lambda a \ b. a \ (\text{sum } b) \ 0$

Примеры:

$$\text{mult } 2 \ 3 \rightarrow_{\beta} 2 \ (\text{sum } 3) \ 0 \rightarrow_{\beta} \quad (30)$$

$$\rightarrow_{\beta} (\text{sum } 3) \ ((\text{sum } 3) \ 0) \rightarrow_{\beta} \quad (31)$$

$$\rightarrow_{\beta} \text{sum } 3 \ 3 \rightarrow_{\beta} 6 \quad (32)$$

$$\text{mult } 0 \ 2 \rightarrow_{\beta} 0 \ (\text{sum } 2) \ 0 \equiv \quad (33)$$

$$\equiv (\lambda s \ z. z) \ (\text{sum } 2) \ 0 \rightarrow_{\beta} 0 \quad (34)$$

$$\text{mult } 1 \ 0 \rightarrow_{\beta} 1 \ (\text{sum } 0) \ 0 \equiv \quad (35)$$

$$\equiv (\lambda s \ z. s \ z) \ (\text{sum } 0) \ 0 \rightarrow_{\beta} \quad (36)$$

$$\rightarrow_{\beta} \text{sum } 0 \ 0 \rightarrow_{\beta} 0 \quad (37)$$

pow := $\lambda a \ b. b \ (\text{mult } a) \ 1$

Примеры:

$$\text{pow } 3 \ 2 \rightarrow_{\beta} 2 \ (\text{mult } 3) \ 1 \rightarrow_{\beta} \quad (38)$$

$$\rightarrow_{\beta} (\text{mult } 3) \ ((\text{mult } 3) \ 1) \rightarrow_{\beta} \quad (39)$$

$$\rightarrow_{\beta} (\text{mult } 3) \ 3 \rightarrow_{\beta} 9 \quad (40)$$

$$\text{pow } 2 \ 0 \rightarrow_{\beta} 0 \ (\text{mult } 2) \ 1 \rightarrow_{\beta} 1 \quad (41)$$

$$\text{pow } 1 \ 3 \rightarrow_{\beta} 3 \ (\text{mult } 1) \ 1 \rightarrow_{\beta} \quad (42)$$

$$\rightarrow_{\beta} (\text{mult } 1) \ ((\text{mult } 1) \ ((\text{mult } 1) \ 1)) \rightarrow_{\beta} \quad (43)$$

$$\rightarrow_{\beta} (\text{mult } 1) \ ((\text{mult } 1) \ 1) \rightarrow_{\beta} 1 \quad (44)$$

Задание 2.3 (16)

Задайте лямбда-терм **isZero**, который возвращает **true**, если подать **0**, и **false** в противном случае.

Решение:**isZero** := $\lambda n. n \ (\text{K false}) \ \text{true}$

Примеры:

$$\text{isZero } 2 \rightarrow_{\beta} 2 \ (\text{K false}) \ \text{true} \rightarrow_{\beta} \quad (45)$$

$$\rightarrow_{\beta} (\text{K false}) \ ((\text{K false}) \ \text{true}) \rightarrow_{\beta} \quad (46)$$

$$\rightarrow_{\beta} (\lambda y. \text{false}) \ ((\text{K false}) \ \text{true}) \rightarrow_{\beta} \text{false} \quad (47)$$

$$\text{isZero } 0 \rightarrow_{\beta} 0 \ (\text{K false}) \ \text{true} \rightarrow_{\beta} \text{true} \quad (48)$$

Задание 2.4 (16)

Скорее всего, ваш любимый язык программирования имеет поддержку функций высших порядков. Представьте, что в вашем языке нет структур, классов и т.д., и закодируйте на нём пары в стиле чистого лямбда-исчисления (*не забывайте про каррирование!*).

Как известно, котика полностью описываются двумя параметрами: именем и мурлычностью. Реализуйте библиотеку функций для работы с котиками, используя пары по Чёрчу как низкоуровневый примитив. Нужно написать функции: для создания котика, просмотра имени и мурлычности по коту. А так же реализовать функцию “почесать”, которая по коту будет возвращать котика повышенной мурлычности.

Если у вас статически типизированный язык программирования:

- Вам полагается дополнительный балл в случае успеха.
- Достаточно поддержать только хранение интов.
- Напишите вручную типы основных сущностей, для остального можно воспользоваться механизмом вывода типов вашего языка: `auto` в C++ , `var` в Java...

Решение:

```

1  module Pair = struct
2      (* Sorry for using struct, small hack. forall quantifier only in fields *)
3      type ('a, 'b) t = { f : 'r. ('a -> 'b -> 'r) -> 'r }
4
5      let make_pair : 'a -> 'b -> ('a, 'b) t = fun a b -> { f = fun f -> f a b }
6      let fst : ('a, 'b) t -> 'a = fun p -> p.f (fun a b -> a)
7      let snd : ('a, 'b) t -> 'b = fun p -> p.f (fun a b -> b)
8  end
9
10 module Church = struct
11     (* Same here *)
12     type t = { f : 'a. ('a -> 'a) -> 'a -> 'a }
13
14     let zero : t = { f = fun s z -> z }
15     let succ : t -> t = fun n -> { f = (fun s z -> s @@ n.f s z) }
16
17     let rec of_int = function
18     | 0 -> zero
19     | n -> succ @@ of_int @@ n - 1
20
21     let to_int n = n.f Int.succ 0
22 end
23
24 module Cat : sig
25     type t
26
27     val make_cat : string -> int -> t
28     val name : t -> string
29     val meowness : t -> int
30     val scratch : t -> t
31 end = struct
```

```

32   open Pair
33   open Church
34
35   type t = (string, Church.t) Pair.t
36
37   let make_cat_impl name meowness : t =
38     make_pair name @@ meowness
39
40   let meowness_impl (cat : t) : Church.t = snd cat
41
42   let make_cat name (meowness : int) : t =
43     make_cat_impl name @@ of_int meowness
44
45   let name (cat : t) : string =
46     fst cat
47
48   let meowness (cat : t) : int =
49     to_int @@ meowness_impl cat
50
51   let scratch (cat) : t =
52     let n = name cat in
53     make_cat_impl n (succ @@ meowness_impl cat)
54 end
55
56 let main =
57   (* Check Church numerals *)
58   assert Church.(2 = to_int @@ succ @@ succ @@ zero);
59   assert Church.(10 = to_int @@ of_int 10);
60
61   (* Check Pair *)
62   assert Pair.(1 = fst @@ make_pair 1 2);
63   assert Pair.(2 = snd @@ make_pair 1 2);
64
65   (* Check cat *)
66   let cat = Cat.make_cat "barsik" 2 in
67   assert Cat.("barsik" = name cat);
68   assert Cat.(2 = meowness cat);
69   let cat = Cat.scratch cat in
70   assert Cat.("barsik" = name cat);
71   assert Cat.(3 = meowness cat);

```

Задание 2.5 (16)

Задайте лямбда-терм **divides3** n , который возвращает **true**, если n делится на 3 без остатка, и **false** в противном случае.

Решение:

Можно использовать пары для создания троек, но получится громоздко, поэтому заведен отдельные

тройки:

```

mkTriple := λx y z f. f x y z
rotTriple := λt. t λx y z f. f z x y
fstTriple := λt. t λx y z. x
divides3 := λn. fstTriple (n rotTriple (mkTriple true false false))

```

Примеры:

$$\text{divides3 } 2 \rightarrow_{\beta} \text{fstTriple (rotTriple (rotTriple (mkTriple true false false)))} \rightarrow_{\beta} \quad (49)$$

$$\rightarrow_{\beta} \text{fstTriple (rotTriple (rotTriple (\lambda f. f true false false)))} \rightarrow_{\beta} \quad (50)$$

$$\rightarrow_{\beta} \text{fstTriple (rotTriple ((\lambda f. f true false false) \lambda x y z f. f z x y))} \rightarrow_{\beta} \quad (51)$$

$$\rightarrow_{\beta} \text{fstTriple (rotTriple ((\lambda x y z f. f z x y) true false false))} \rightarrow_{\beta} \quad (52)$$

$$\rightarrow_{\beta} \text{fstTriple (rotTriple (\lambda f. f false true false))} \rightarrow_{\beta} \quad (53)$$

$$\rightarrow_{\beta} \text{fstTriple (\lambda f. f false false true)} \rightarrow_{\beta} \quad (54)$$

$$\rightarrow_{\beta} (\lambda f. f false false true) \lambda x y z. x \rightarrow_{\beta} \quad (55)$$

$$\rightarrow_{\beta} (\lambda x y z. x) \text{ false false true} \rightarrow_{\beta} \text{false} \quad (56)$$

$$\text{divides3 } 3 \rightarrow_{\beta} \text{fstTriple (rotTriple (rotTriple (rotTriple (mkTriple true false false))))} \rightarrow_{\beta} \quad (57)$$

$$\rightarrow_{\beta} \text{fstTriple (rotTriple (rotTriple (rotTriple (\lambda f. f true false false))))} \rightarrow_{\beta} \quad (58)$$

$$\rightarrow_{\beta} \text{fstTriple (rotTriple (rotTriple (\lambda f. f false true false)))} \rightarrow_{\beta} \quad (59)$$

$$\rightarrow_{\beta} \text{fstTriple (rotTriple (\lambda f. f false false true))} \rightarrow_{\beta} \quad (60)$$

$$\rightarrow_{\beta} \text{fstTriple (\lambda f. f true false false)} \rightarrow_{\beta} \text{true} \quad (61)$$

Задание 2.6 (26)

Задайте лямбда-терм **pred** n , который находит предыдущее число Чёрча.

Подсказка:

```

def pred(n):
    prd = 0
    cur = 0
    for i in range(0, n):
        prd = cur
        cur = cur + 1
    return prd

```

Решение:

```

pred := λn. fst (n (λp. pair (snd p) (suc (snd p)))) (pair 0 0)

```

Примеры:

$$\mathbf{pred\ 2} \rightarrow_{\beta} \mathbf{fst} ((\lambda p. \mathbf{pair} (\mathbf{snd}\ p) (\mathbf{suc} (\mathbf{snd}\ p))) ((\lambda p. \mathbf{pair} (\mathbf{snd}\ p) (\mathbf{suc} (\mathbf{snd}\ p))) (\mathbf{pair\ 0\ 0}))) \rightarrow_{\beta} \quad (62)$$

$$\mathbf{pred\ 2} \rightarrow_{\beta} \mathbf{fst} ((\lambda p. \mathbf{pair} (\mathbf{snd}\ p) (\mathbf{suc} (\mathbf{snd}\ p))) (\mathbf{pair\ 0} (\mathbf{suc\ 0}))) \rightarrow_{\beta} \quad (63)$$

$$\mathbf{pred\ 2} \rightarrow_{\beta} \mathbf{fst} ((\lambda p. \mathbf{pair} (\mathbf{snd}\ p) (\mathbf{suc} (\mathbf{snd}\ p))) (\mathbf{pair\ 0\ 1})) \rightarrow_{\beta} \quad (64)$$

$$\mathbf{pred\ 2} \rightarrow_{\beta} \mathbf{fst} (\mathbf{pair\ 1} (\mathbf{suc\ 1})) \rightarrow_{\beta} \quad (65)$$

$$\mathbf{pred\ 2} \rightarrow_{\beta} \mathbf{fst} (\mathbf{pair\ 1\ 2}) \rightarrow_{\beta} \mathbf{1} \quad (66)$$

$$\mathbf{pred\ 0} \rightarrow_{\beta} \mathbf{fst} (\mathbf{pair\ 0\ 0}) \rightarrow_{\beta} \mathbf{0} \quad (67)$$